Lab 5 实验报告

实验要求

CPU 需要支持以下指令并通过测试：

实现指令：`MRET`, `ECALL`

实现 MMU，支持 Sv39 页表

实验过程

这一小节中，所有 AI 帮助完成的部分都进行了标注

参考

- AI(DeepSeek)
- 复旦大学25春计算机组成与结构(H)课程项目五级流水线CPU

Step 0

添加了特权级别寄存器，完成了 `ecall` 和 `mret` 指令及相关的 `csr` 寄存器的修改。

Step 1

参照 复旦大学25春计算机组成与结构(H)课程项目五级流水线CPU，完成了 `mmu`。

调试过程

这一小节中，所有 AI 帮助完成的部分都进行了标注

问题 1

在 `M` 模式下，连续 `load` 出错。

解决 1

发现是在生成 `stall` 信号时，没有考虑 `load` 指令对寄存器的使用情况，这也说明在之前的测试中，没有连续两条 `load` 指令，且前一条指令的 `rd` 与后一条指令的 `rs1` 相同。

问题 2

遇到 `mret` 时，没有及时处理特权级别转换，导致取指的地址出现问题。

解决 2

将特权级别修改提到了 `ex_stage`。

问题 3

将特权级别提前后，发现当遇到特权级别变化的指令时，正在提交的指令与特权级别不相符。

解决 3

为每一个指令赋一个单独的记录特权的信号。

问题 4

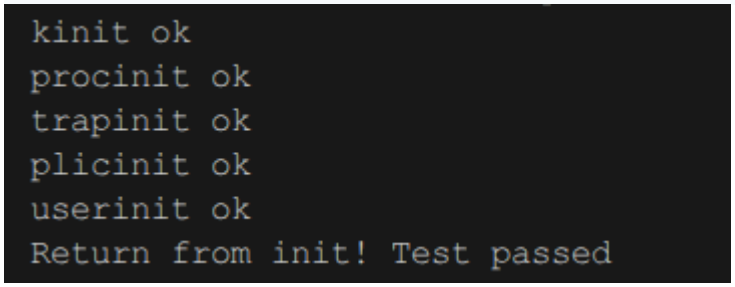
遇到 `ecall` 指令时，由于修改寄存器需要用到 `ecall` 指令之前的特权级别，但对 `csr` 寄存器的修改滞后于特权级别的修改，因此引入 `prev_priv` 记录之前的特权级别，并且采用如下更新逻辑，保证 `prev_priv` 逻辑正确。

```
u64 priv_change_pc;
    always_ff @(posedge clk) begin
        if (rst) begin
            current_priv <= 2'b11;
            prev_priv <= 2'b11;
            priv_change_pc <= 64'b0;
        end else if (priv_we && id_ex_reg.pc != priv_change_pc) begin
            prev_priv <= current_priv;
            current_priv <= next_priv;
            priv_change_pc <= id_ex_reg.pc;
        end
    end
```

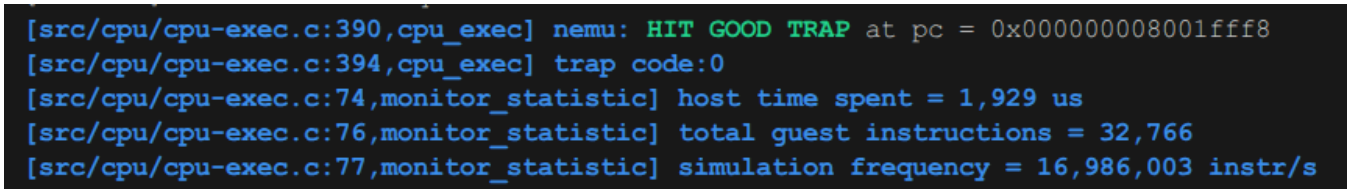
以下是测试结果：

Lab 5

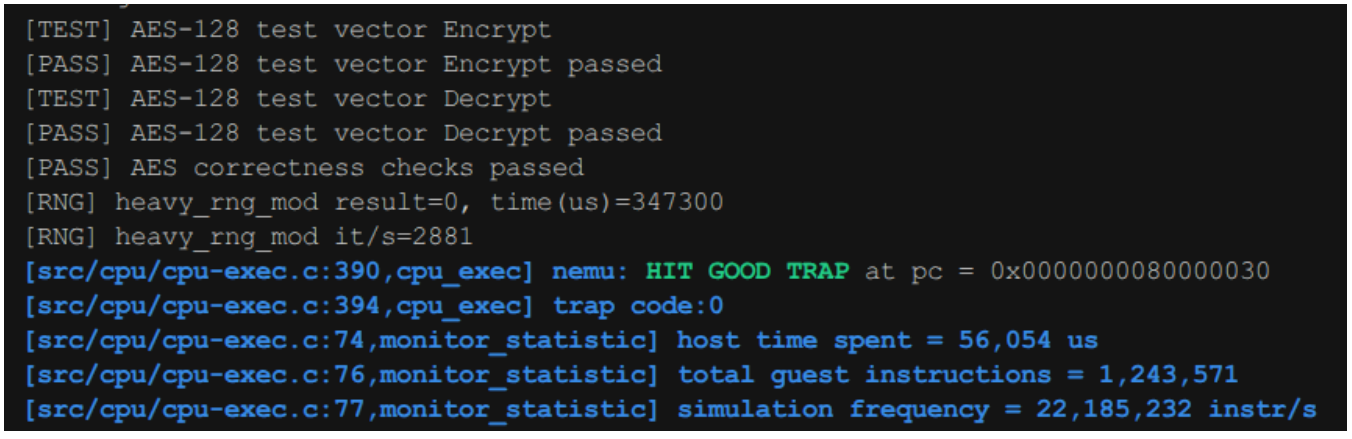
```
kinit ok
procinit ok
trapinit ok
plicinit ok
userinit ok
Return from init! Test passed
```



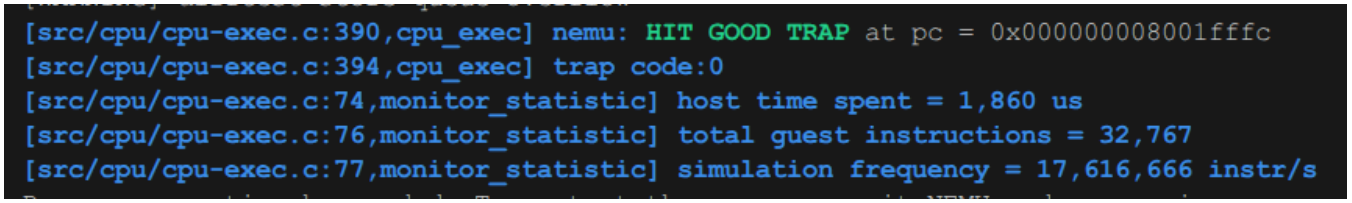
Lab 4



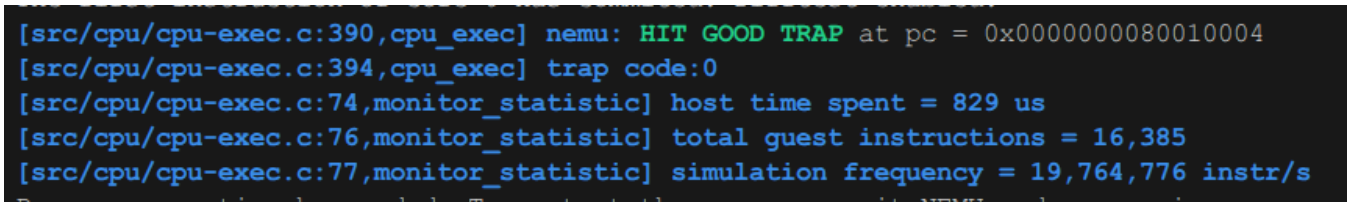
Lab 3



Lab 2



Lab 1


[src/cpu/cpu-exec.c:390,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000080010004
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 829 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 16,385
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 19,764,776 instr/s